Kestrel Microfinance Bank: loan platform on AWS

Design document, 16 October 2026. Prepared by Ismail Oyeleke. Written for readers who are not engineers; the technical detail is in the repository, and each decision links to its decision record (ADR).

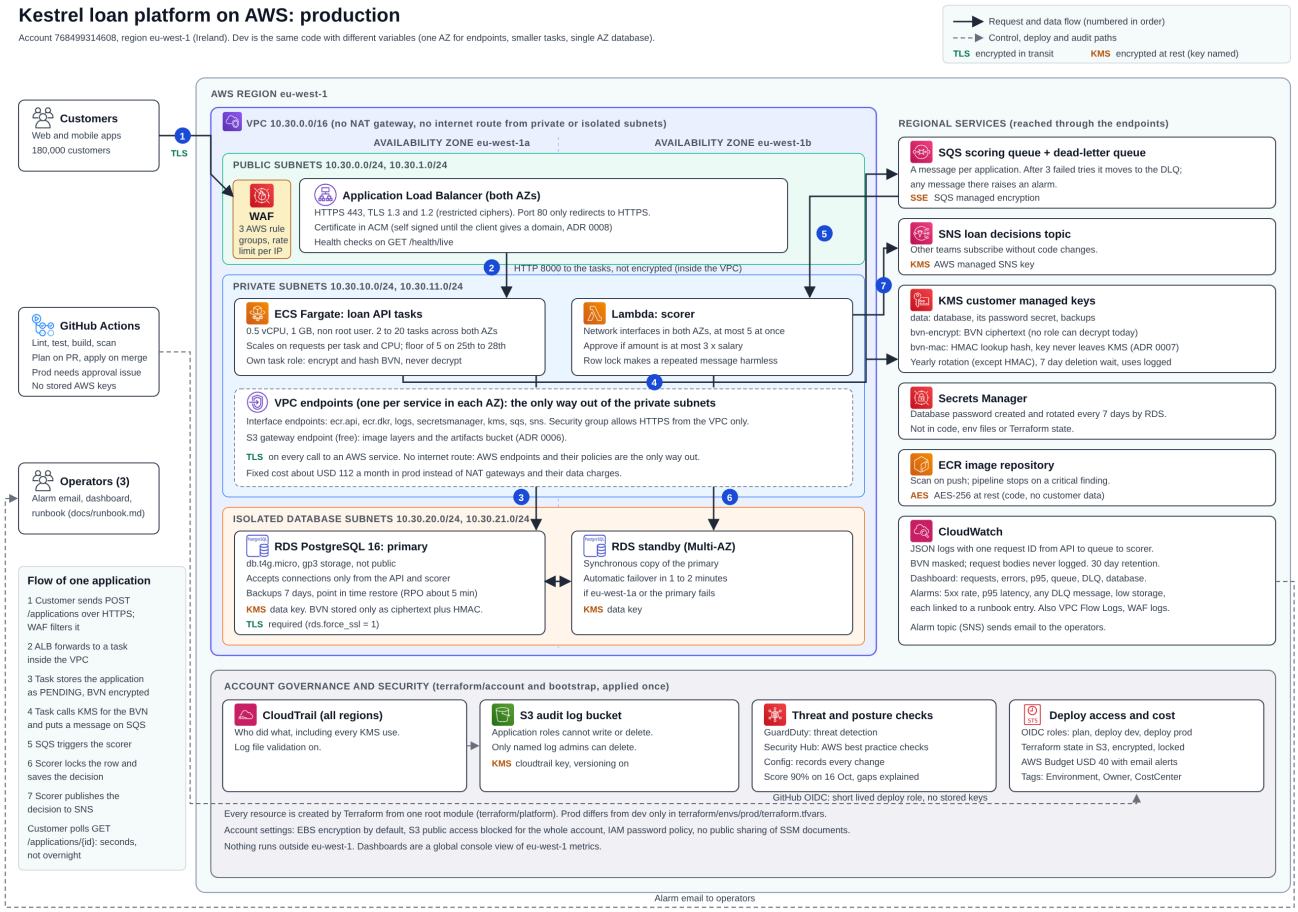
1. Summary

Kestrel's loan platform today is one server and one database in a local data centre. It goes down almost every month when salaries land, scores loans overnight, and has no audit trail. We built a replacement on AWS in Ireland (eu-west-1) and tested it at payday load.

What Kestrel asked for	What we delivered	Proof
No more payday outages	The platform adds servers by itself as traffic grows, and starts with more on the 25th to the 28th. At 25 times normal traffic, no request failed and responses took under a tenth of a second	Load test, section 5
Loan decisions in minutes, not overnight	Each application is scored seconds after it arrives	Load test: the scoring queue never built up
Evidence for auditors	AWS account activity is recorded in a log nobody in the application can change. Public traffic, the database and backups are encrypted, with the BVN given extra protection	Section 6
RPO 15 minutes, RTO 2 hours	In a real restore drill on a small database: newest restore point 4 min 44 s old, service stable 21 minutes after the restore started	Section 7
Under USD 1,500 a month	About USD 273 a month at Kestrel's real size, with a forecast	Section 8
Nigeria Data Protection Act 2023, and Cape Town or Europe	Stay in Europe for now, with the legal steps below; plan for Nigeria hosting of payment data from 2027	Section 9

2. How it works

See the architecture diagram (`docs/design/architecture.png`). In plain terms:



1. A customer's phone or browser sends a loan application over an encrypted connection.
2. A web firewall blocks known attacks and anyone sending too many requests. A load balancer spreads the rest across the application servers in two separate data centres (availability zones) in Ireland.
3. The application server saves the application. Before saving, it encrypts the customer's BVN with a key held in AWS's key service. Nobody reading the database, a backup or a stolen copy can see a BVN.
4. The application puts a "please score this" message on a queue and answers the customer at once.
5. A separate scoring function picks up the message within seconds, decides, saves the decision, and announces it so other systems (SMS, email, reporting) can react without code changes.
6. The customer's app checks back and sees the decision.

The servers and database have no route to the internet. They reach AWS services only through private connections (VPC endpoints), so the paths data could take out are limited to those AWS services and their access policies.

3. Key decisions

Decision	Why	What it costs us	Record
One AWS account, dev and prod kept apart by network, keys, roles and state	Fast and cheap for a five day build	Weaker than separate accounts: our deployment roles can reach both. For Kestrel: separate accounts	ADR 0001

Decision	Why	What it costs us	Record
Everything built from code (Terraform), with plan on every change and a human approval before production	Repeatable, reviewable, nothing changed by hand	Changes take a pull request	ADR 0002, 0009
No stored passwords or keys for the build pipeline: it signs in with short lived tokens	The incident six months ago was a password in Git	None	ADR 0003
Private connections to AWS services instead of an internet gateway for servers	Data cannot leave; no surprise data charges	About USD 40 a month more than the cheapest option	ADR 0006
Scoring on a queue with a separate function	Decisions in seconds; a burst of applications waits safely in the queue; a failed message goes to a holding queue and raises an alarm	A little more to monitor	ADR 0005
A repeated message can never produce two decisions	Queues can deliver a message twice	None	ADR 0005
BVN encrypted, plus a one way fingerprint for searching	Meets "never readable in the database"; still allows "find by BVN"	The application cannot decrypt a BVN; a future verification service would get its own audited permission	ADR 0007
HTTPS with our own certificate until Kestrel provides a domain	No domain was available	Customers' apps must trust our certificate; a free trusted certificate replaces it the day a domain exists	ADR 0008
Bigger servers in production and a higher minimum from the 25th to the 28th	The payday spike arrives faster than new servers can start	About USD 7 a month for the payday minimum	ADR 0010
Database copy in a second data centre (Multi-AZ)	If one data centre fails, the copy takes over in a minute or two	Doubles the database cost (about USD 15 a month more)	Section 7

4. What we cut, and why

Cut or deferred	Why	What Kestrel should do
Separate AWS accounts and organisation wide guard rails (SCPs)	One account for a short assessment budget	Set up AWS Organizations with dev, prod and log accounts before go-live
Locked log storage (S3 Object Lock)	Stretch goal; cannot be undone during a short build	Turn on in the log account with a retention agreed with auditors
Tracing each request across services (X-Ray)	Stretch goal; logs already carry one request ID end to end	Add if diagnosis takes too long
Copy of backups to a second region	Stretch goal	Add if the bank needs to survive the loss of the whole Ireland region
A real domain and trusted certificate	Kestrel has none yet	Provide one; the change is small (ADR 0008)
Some Security Hub controls (load balancer access logs, deletion protection, hardware MFA for the root user)	Cost or would block the daily teardown	Turn on for production (docs/security-findings.md)

Cut or deferred	Why	What Kestrel should do
Part B of the incident challenge	Removed by the hiring contact on 15 Oct	None

5. Payday load test

We defined normal as 2 requests a second (3,600 customers in the busiest hour) and tested 25 times that: 50 a second for 5 minutes, after a 2 minute ramp. Result: 19,602 requests, none failed, 95% answered within 0.058 seconds against a 1 second target. The servers added capacity by themselves from 2 to 5. One lesson: they kept adding past what was needed, which hit an AWS account limit and made the next update fail and roll back safely. The fix is a free limit increase and no deployments from the 25th to the 28th. Details in `docs/load-test.md`.

6. Security and audit evidence

- **Who did what:** AWS management activity in every region is recorded (CloudTrail) in a storage bucket that the application cannot write to or delete from, encrypted with its own key. Application logs carry a request ID per call. Reads of stored data and SQL activity are not in this trail; they need separate data event and database audit logging.
- **Encryption:** customer traffic to the load balancer and connections to the database use TLS 1.2 or 1.3. The hop from the load balancer to the application inside the private network is plain HTTP today (a known risk, section 11). At rest, everything is encrypted; the database, its backups and the BVN use keys Kestrel controls.
- **No passwords in code:** the database password is created and changed every 7 days by AWS and never appears in code or files.
- **Continuous checks:** threat detection (GuardDuty) and AWS's security best practice checks (Security Hub) are on. Score 90% on 16 October; every open item is explained and the free ones are fixed (`docs/security-findings.md`).
- **Alarms:** server errors, slow responses, no healthy servers, scoring falling behind, any failed scoring message and low database storage each send an email, and each links to a runbook entry the three person team can follow (`docs/runbook.md`).

7. Recovery: RPO and RTO

On 16 October we deliberately "lost" data, restored the production database to a moment before the loss, pointed the application at the restored copy and checked the right data came back. Measured on a small database: the service was **stable 21 minutes after the restore was requested (target 2 hours)**, and the first data check passed at 22 minutes 50 seconds. When the restore was requested, the newest point AWS could restore to was **4 minutes 44 seconds old (target 15 minutes)**; the drill chose an earlier point, 11 minutes 27 seconds back, just before the deliberate loss. The spare time covers noticing the problem and deciding to restore. A real recovery also has to pause new applications, check messages already in the queue against the restored data, and confirm new applications are scored. A bigger database would take longer to restore, so the drill should be repeated every quarter. Details in `docs/restore-drill.md`.

The database also has a live copy in a second data centre. If the first one fails, AWS switches over in a minute or two, with no data lost; this is far inside the 2 hour target. If the copy were dropped to save money, a data centre failure would mean a restore like the drill: about 21 minutes, losing the writes made since the newest restore point (typically about 5 minutes).

8. Cost and forecast

Estimated on 16 October with the AWS Pricing Calculator for Kestrel's real size (180,000 customers, about 2 million requests a month, payday at 25 times normal), in Ireland, on demand prices, no upfront payment: https://calculator.aws/#/estimate?id=7703547414a23b80957496096e1cd9d9024d8d38

(screenshot `docs/evidence/pricing-calculator-summary.png`). Lambda is priced without its free tier, to stay on the safe side.

Total: USD 273.48 a month, USD 3,281.76 a year. That is 18% of the USD 1,500 monthly limit.

What it pays for	AWS service	USD a month
Private connections to AWS services (7 endpoints in 2 data centres)	VPC interface endpoints	112.52
Application servers: 2 always on, 5 from the 25th to the 28th, up to 10 for a few peak hours	Fargate	44.14
Database with a live copy in a second data centre	RDS PostgreSQL, db.t4g.micro Multi-AZ, 20 GB	30.63
Security checks and audit: change history, best practice checks, threat detection, activity log	Config, Security Hub, GuardDuty, CloudTrail	37.44
Load balancer	Application Load Balancer	about 19
Web firewall	WAF	10.20
Encryption keys and the database password	KMS, Secrets Manager	about 11
Logs, dashboard and alarms	CloudWatch	6.34
Loan scoring and messages	Lambda, SQS, SNS	2.23
Stored application images	ECR	0.20
Total		**273.48**

The load balancer and keys lines are rounded; the calculator total is exact. Not included, because each is under a dollar or not known yet: internet data out (small responses, inside the free 100 GB), log storage after ingestion (about USD 0.30), storage for the log buckets, tax, and an AWS support plan (Business support starts at USD 100 a month and is worth considering for a bank).

Forecast. The fixed part (private connections, database, load balancer, firewall) does not grow with customers. The servers and requests grow roughly in line with traffic. About USD 70 of today's bill grows with traffic (servers, request charges, log volume, threat detection); the other USD 200 is fixed. At double today's customers the bill would be about USD 345. The bill is not what limits growth first: the small database is. At about 10 times today's traffic the database would need a larger size (our estimate: roughly USD 250 a month more, not in the calculator link), which still keeps the total near USD 1,150, under the limit. These forecasts are our arithmetic from the calculator lines, not a second calculator run.

Three ways to lower the bill

1. **Commit for a year.** A one year Compute Savings Plan for the servers and a one year reserved database instance cut those prices by roughly a fifth to a third for a steady workload like this one. Saving: about USD 15 to 20 a month. Risk: low; the commitment is small.
2. **Use AWS Graviton (ARM) processors** for the servers and the scoring function. AWS prices Graviton Fargate about 20% below Intel, and it uses less energy. Saving: about USD 9 a month now, growing with traffic. Cost: build the image for ARM and rerun the load test.
3. **Put the private connections in one data centre instead of two.** Saving: about USD 56 a month, the biggest single cut. Risk: if that data centre fails, servers in the other one lose their path to AWS services until it recovers. We do not recommend this for a bank, but it is the lever if money gets tight.

Savings 1 and 2 are estimates from AWS's published discounts and should be priced in the calculator before committing.

9. Cape Town (af-south-1) or Europe (eu-west-1)?

Recommendation: stay in Ireland for this platform today, and plan now for Nigerian hosting of payment data. This is an engineering view, not legal advice; Kestrel's data protection officer and lawyers should confirm it.

What the Nigeria Data Protection Act 2023 says. It does not require personal data to stay in Nigeria. It allows a transfer abroad when the destination gives adequate protection, or on other grounds such as the customer's informed consent or a contract with the customer (sections 41 to 43). The Nigeria Data Protection Commission's General Application and Implementation Directive (2025) lists how adequacy is judged, including an effective data protection law and an independent regulator. Ireland is under the EU GDPR, one of the strictest regimes, so the case for adequacy is strong. South Africa (POPIA) also has a law and a regulator. **Both regions are a transfer out of Nigeria**: Cape Town is not "local" in law.

What may change in 2027. Press reports (Legal500, Techpoint Africa, BusinessDay) describe a June 2026 Central Bank of Nigeria circular requiring payment transaction data generated in Nigeria to be stored and managed in Nigeria from 1 January 2027, and list microfinance banks among the institutions covered. We have not read the circular itself, so Kestrel's compliance team must confirm it. Loan applications are not payment transactions, but loan payouts and repayments would be. Neither Ireland nor Cape Town meets that rule. The AWS option in Nigeria is the Lagos Local Zone, an extension of the Cape Town region that runs servers, containers and load balancers, but not managed databases.

	Ireland (chosen)	**Cape Town**
NDPA transfer	Needs adequacy or another ground; strong case (GDPR)	Needs adequacy or another ground (POPIA)
CBN payment data from 2027 (reported)	Does not meet it	Does not meet it either; but it is the parent region of the Lagos Local Zone
Services	Every service we use	Must be switched on per account; most services we use exist, but check each one before moving
Cost	Lower	Generally higher for the same services
Distance to Lagos	Measure from Kestrel's network before deciding	Measure; not automatically faster, because traffic may route through Europe

What Kestrel should do: (1) record the legal ground for the transfer to Ireland and tell customers in the privacy notice; (2) register with the Commission if it is a data controller of major importance; (3) get a legal reading of whether loan data falls under the CBN circular; (4) if payment data is added, host that part in the Lagos Local Zone or a Nigerian provider, which this design supports because everything is built from code.

10. Well-Architected self review

Pillar	**Strongest point**	**Weakest point**
Operational excellence	Everything is code, every change is planned on a pull request, and every alarm links to a runbook step	One account and one person's GitHub approval; no on-call rota or tested incident drill with the client team yet
Security	No internet route from the servers; BVN encrypted with a key the application cannot use to decrypt; no stored credentials anywhere	Deployment roles are full administrators of the shared account (ADR 0001, 0003)
Reliability	Two data centres for servers and database; automatic rollback of a bad release; measured restore in 21 minutes	The AWS Fargate limit of 12 processors can block a release after a big scale out; needs a limit increase before go-live

Pillar	Strongest point	Weakest point
Performance efficiency	50 requests a second answered in 0.058 seconds with servers at a third of their capacity	Scaling overshoots (5 needed, 13 started) because new servers take minutes to count as ready
Cost optimisation	Within budget at Kestrel's size, with extra capacity paid for only on payday days	Fixed private connection cost (about USD 112 a month) is the largest single line at today's traffic
Sustainability	Scales down after payday; serverless scoring uses nothing when idle	Servers run on Intel (x86); AWS Graviton processors use less energy for the same work

11. Main risks and next steps

1. Ask AWS for a Fargate processor limit of at least 40 (free) and freeze deployments from the 25th to the 28th.
2. Move to separate AWS accounts before go-live.
3. Provide a domain so customers get a publicly trusted certificate.
4. Ask Kestrel's compliance team to confirm the CBN circular and the lawful basis for processing data outside Nigeria. Both Ireland and Cape Town are outside Nigeria.
5. Repeat the restore drill every quarter and the load test before each payday season.

Known limitations, stated honestly:

- **Saving and queueing are two steps.** The API saves an application, then sends it to the scoring queue. A crash or a lost reply between the two can leave an application unscored or removed. The fix is a transactional outbox (save the application and its queue job together, then publish).
- **Notifications are at least once.** A decision is never stored twice, but a notification can repeat with a different timestamp. Subscribers must ignore repeats by application ID.
- **Deploy roles are full administrators** of the shared account, so dev credentials could change prod (ADR 0001, 0003).
- **The application uses the database's main user.** Separate users for the API, the scorer and schema changes would limit what a compromised task could do.
- **The load balancer to application hop is plain HTTP** inside the private network.
- **The load test checked responses, not final decisions.** It proves responsiveness at the payday peak, not how long scoring took under that load.